

Chapter 1. 에이전트의 최소 구성 요소

Part I. AI 에이전트란 무엇인가

이제부터 3개 챕터 동안, "AI 에이전트"라는 말이 실제 코드에서 정확히 무엇을 가리키는 지부터 시작해(1장), 그 최소 단위 하나가 어떻게 동작하는지 한 줄씩 따라가고(2장), 마지막으로 그게 실제로 어떻게, 왜 실패하는지(3장)까지 확인한다. Part II부터는 "여러 에이전트가 어떻게 협업하는가"로 넘어가므로, Part I은 그 전에 필요한 **에이전트 하나**에 대한 그림을 완성하는 구간이다.

이 챕터에서 배우는 것

- 업계에서 "AI 에이전트"를 설명할 때 공통으로 등장하는 요소(추론 엔진·도구·메모리·오케스트레이션)와 자율성 스펙트럼
- "AI 에이전트"라는 말이 Book-forge 실제 코드에서는 무엇을 가리키는지
- Book-forge의 **LLM** Protocol이 왜 OpenAI/Anthropic/Ollama를 똑같은 인터페이스로 감싸는지
- 에이전트를 최소로 정의하면 무엇이 남고 무엇이 빠지는지

이런 분이 먼저 읽으면 좋습니다: "에이전트"라는 단어를 여러 문서에서 다르게 쓰는 걸 보고 혼란스러웠던 분. 이 챕터는 정의를 논쟁하는 대신, 실제로 동작하는 코드 하나를 기준점으로 삼는다.

1.1 일반적으로 "AI 에이전트"란 무엇을 가리키는가

Book-forge의 정의로 들어가기 전에, 업계에서 "AI 에이전트"라는 말이 대략 어떤 공통 요소를 가리키는지부터 정리해보자. 이렇게 정리해두면, 이후 Book-forge의 선택이 이 스펙트럼 어디에 있는지 판단할 기준이 생긴다.

공통으로 등장하는 네 가지 구성 요소. 문헌마다 용어는 다르지만, "AI 에이전트"를 설명할 때 대체로 이 네 요소의 조합으로 이야기한다.

요소	하는 일	Book-forge의 대응(1~2장에서 확인)
추론 엔진 (reasoning core)	입력을 이해하고 다음 행동을 결정한다 — 보통 LLM이 이 역할을 맡는다	<code>LLM</code> Protocol의 <code>generate()</code> (§ 1.2)
도구 (tools)	에이전트가 세계에 영향을 미치는 수단 — 함수 호출, 파일 쓰기, 웹 검색 등	이 책의 개별 에이전트에는 거의 없다 (§ 1.5) — 파일 쓰기는 별도 <code>@tool_guard</code> 축(12장)이 담당
메모리 (memory)	이전 상호작용이나 축적된 지식을 다음 판단에 반영하는 저장소	<code>conversation_history</code> (7장), <code>KnowledgeStore</code> (7장)
오케스트레이션 (orchestration/planning)	여러 단계·여러 에이전트를 어떤 순서로 실행할지 결정하는 상위 로직	<code>new_cmd.py</code> 의 <code>new()</code> (4장), 승인 루프(6장)

자율성은 스펙트럼이다. 이 네 요소를 얼마나 갖췄는가에 따라 "에이전트"라 불리는 것들은 실제로 매우 다른 자율성 수준에 있다. 한쪽 끝에는 "프롬프트 하나 → 응답 하나"로 끝나는 단순 래퍼가 있다. 반대쪽 끝에는 스스로 목표를 여러 하위 작업으로 쪼개고, 어떤 도구를 쓸지 매 단계 스스로 판단하고, 실패하면 다른 방법을 시도하는 완전 자율 에이전트가 있다. 대부분의 실무 시스템은 이 둘 사이 어딘가에 있다. **그리고 그 위치를 정확히 아는 것이, "에이전트"라는 한 단어보다 훨씬 중요한 정보다.** 3장(§ 3.6)에서 다시 확인하겠지만, 어떤 방어 메커니즘이 필요한가는 정확히 이 위치에서 결정된다. 도구를 안 쓰는 에이전트에는 도구 오남용 방어가 필요 없고, 반복 루프가 없는 에이전트에는 무한 루프 방어가 필요 없다.

단일 에이전트와 멀티 에이전트. 위 네 요소는 에이전트 하나의 내부 구조를 말한다. 그런데 실무에서는 에이전트 하나로 끝나는 경우보다, 여러 에이전트가 각자 다른 역할을 맡아 협업하는 경우가 더 흔하다(검색 전담, 코드 작성 전담, 검토 전담처럼 역할을 나누는 방식). "협업"이 정확히 어떤 형태를 띠는지는 시스템마다 다르다. 정해진 순서로 결과를 넘기기만 하는 파이프라인일 수도 있고, 여러 에이전트가 동시에 같은 문제를 보고 토론하는 구조일 수도 있다. Part II(4~7장) 전체가 이 "여러 에이전트의 협업"이 Book-forge에서 실제로 어떤 네 가지 형태로 나타나는지 다룬다.

이 책은 위 개념들에 대한 논쟁(어디까지가 "진짜 에이전트"인가 같은)에 끼어들지 않는다. 대신 Book-forge가 실제로 코드로 구현한 선택에서 출발해, 그 선택이 왜 이 스펙트럼의 그 위치에 있는지를 실제 코드로 보여준다. 가장 작은 단위(에이전트 하나)부터 시작해보자.

1.2 가장 작은 정의부터 시작한다

"AI 에이전트"라는 말은 문맥마다 다른 것을 가리킨다. 어떤 글은 자율적으로 도구를 골라 쓰는 시스템을 뜻하고, 어떤 글은 단순히 "LLM을 호출하는 함수"를 그렇게 부른다. 이 책은 Book-forge가 실제로 코드로 구현한 정의에서 출발한다. 그 출발점은 `src/book_forge/llm/provider.py` 다.

```
@runtime_checkable
class LLM(Protocol):
    """모든 provider 구현체가 만족해야 하는 최소 인터페이스."""

    model: str

    def generate(
        self, prompt: str, *, system: Optional[str] = None, max_tokens: int = 4000
    ) -> str: ...
```

이게 전부다. 문자열 하나(`prompt`)를 받아 문자열 하나(`generate()`의 반환값)를 돌려주는 것, 그게 전체 계약이다. Book-forge의 모든 에이전트는 이 인터페이스 위에 얹게 쌓은 껍데기일 뿐이다. "에이전트가 무엇을 하는가"는 이 `generate()` 호출 앞뒤에 무엇을 붙이느냐로 결정된다. 어떤 프롬프트를 조립해 넘기는지, 응답을 어떻게 파싱하는지, 그 결과로 무엇을 하는지가 에이전트마다 다를 뿐이다.

1.3 세 개의 구현체, 하나의 계약

`LLM` Protocol을 실제로 만족하는 구현체는 세 개다.

구현체	실제로 호출하는 것	기본 모델
<code>OpenAILLM</code>	OpenAI Chat Completions API	<code>gpt-5-nano</code>

구현체	실제로 호출하는 것	기본 모델
AnthropicLLM	Anthropic Messages API	claude-haiku-4-5-20251001
OllamaLLM	로컬 <code>http://localhost:11434/api/generate</code>	llama3.2

세 클래스의 `generate()` 시그니처는 동일하지만, 내부에서 실제로 하는 일은 서로 다른 HTTP 요청 형식을 조립하는 것이다. 예를 들어 `OllamaLLM.generate()` 는 이렇게 페이로드를 만든다.

```
payload = {
    "model": self.model,
    "prompt": prompt,
    "system": system or "",
    "stream": False,
    "think": False, # 추론 모델이 "생각"만 하고 답을 안 내는 문제 방지
    "options": {"num_predict": max_tokens},
}
response = requests.post(f"{self._base_url}/api/generate", json=payload, timeout=180)
```

`think: False` 옵션 하나가 이 책 전체에서 반복해 등장할 "에이전트는 왜 실패하는가"(3장)의 첫 사례다. `qwen3.6:35b-mlx` 같은 추론(thinking) 모델은 사고 과정을 별도 필드에 담고 최종 응답(`response`)은 비워둘 수 있다. `num_predict` (토큰 예산)를 추론에 다 써버리면 챗터 파일이 통째로 빈 채 저장되는 사고가 실제로 있었다(`provider.py`의 주석에 이 실측 경위가 그대로 남아 있다). `think: False` 는 그 사고 과정을 건너뛰고 바로 답을 채우게 강제하는, 코드 한 줄로 막은 실패 사례다.

나머지 두 구현체(`OpenAILLM` · `AnthropicLLM`)도 `provider.py` 에서 그대로 확인할 수 있다. 각자 다른 SDK(`openai` / `anthropic` 패키지)를 감싸지만, 바깥에서 보이는 모양은 정확히 같은 `generate(prompt, *, system=None, max_tokens=4000) → str` 이다.

```
class OpenAILLM:
    def __init__(self, api_key: str, model: str = DEFAULT_OPENAI_MODEL) -> None:
        from openai import OpenAI # lazy import

        self._client = OpenAI(api_key=api_key)
        self.model = model
```

```

def generate(
    self, prompt: str, *, system: Optional[str] = None, max_tokens: int = 4
000
) -> str:
    messages = []
    if system:
        messages.append({"role": "system", "content": system})
    messages.append({"role": "user", "content": prompt})
    response = self._client.chat.completions.create(
        model=self.model, messages=messages, max_tokens=max_tokens,
    )
    return response.choices[0].message.content or ""

class AnthropicLLM:
    def __init__(self, api_key: str, model: str = DEFAULT_ANTHROPIC_MODEL) -> None:
        from anthropic import Anthropic # lazy import

        self._client = Anthropic(api_key=api_key)
        self.model = model

    def generate(
        self, prompt: str, *, system: Optional[str] = None, max_tokens: int = 4
000
    ) -> str:
        response = self._client.messages.create(
            model=self.model, max_tokens=max_tokens,
            system=system or "", messages=[{"role": "user", "content": prompt}],
        )
        return "".join(block.text for block in response.content if hasattr(block, "text"))

```

세 구현체를 나란히 놓고 보면 "Protocol을 만족한다"는 말이 코드로 무엇을 뜻하는지 뚜렷해진다. `OpenAILLM` 은 `messages` 리스트(역할별 딕셔너리)를 조립하고, `AnthropicLLM` 은 `system` 을 별도 인자로 넘기고, `OllamaLLM` 은 순수 JSON 페이로드를 만든다. 세 API의 요청 형식은 서로 판이하게 다르지만, 세 클래스 모두 `prompt` 와 `system` 을 받아 문자열 하나를 돌려준다는 점만은 동일하다. 이 책의 나머지 부분(2장부터)은 "LLM을 어떤 provider가 서비스하는지"를 단 한 번도 신경 쓰지 않는다. 그 이유는 바로 여기, `provider.py` 안에서 그 차이가 이미 흡수됐기 때문이다.

1.4 provider 선택 — 환경변수 하나가 전체 파이프라인을 바꾼다

`create_llm()` 팩토리 함수가 어떤 구현체를 만들지 결정한다.

```
def create_llm(provider: Optional[str] = None, model: Optional[str] = None) -> LLM:
    resolved = (provider or os.environ.get("LLM_PROVIDER") or "ollama").lower()
    ...
```

우선순위는 명시적 인자 → `LLM_PROVIDER` 환경변수 → 기본값 `"ollama"` 순이다. 기본값을 클라우드가 아니라 로컬로 둔 이유는 코드 주석에 명시돼 있다. "API 키 없이 바로 동작해 오프라인/로컬 개발을 1급 경로로 만든다." 이 한 줄의 설계 결정이 Book-forge 전체의 성격을 규정한다. 그래서 이 책의 모든 실측 예제도 API 키 없이 로컬 Ollama로 재현할 수 있다.

 **개발자 TIP:** 에이전트를 새로 만들 때 "이 함수가 Protocol을 만족하는가"만 확인하면 provider 걱정 없이 코드를 짤 수 있다.

`planner.py` · `chapter_drafter.py` · `chat_agent.py` 어디에도 `if provider == "openai"` 같은 분기가 없다 — 전부 `llm.generate(...)` 한 줄만 호출한다.

1.5 이 정의에 일부러 넣지 않은 것

§ 1.1의 네 요소로 돌아가 보면, 이 최소 정의에는 "도구를 자율적으로 선택한다"거나 "여러 단계를 스스로 계획한다"는 요건이 없다. Book-forge의 개별 에이전트 (`propose_plan()`, `design_toc()` 등)는 정확히 한 번 `generate()` 를 호출하고 끝난다. 도구 호출도, 반복 루프도 없다. **에이전트 하나만 놓고 보면 자율성 스펙트럼의 가장 단순한 쪽 끝에 있다.** 오케스트레이션(여러 에이전트를 어떤 순서로 부를지)과 메모리(무엇을 다음 에이전트에 넘길지)는 에이전트 자신이 아니라 그 바깥(`new_cmd.py`, `KnowledgeStore`)이 담당한다. Part II(4~7장)가 정확히 이 "바깥" 구조를 다룬다.

그런데도 이 책은 이들을 "에이전트"라 부른다. 왜 그럴까. 다음 장에서 확인하겠지만, 이 함수들을 실제 시스템으로 만드는 것은 LLM 호출 자체가 아니라 **그 호출 앞뒤에 계층**

(`@agent_eval`)이 붙어 있다는 사실이다. "에이전트"와 "그냥 LLM을 부르는 함수"를 가르는 경계가 바로 이 책의 3부·4부가 다루는 Harness Engineering이다.

직접 해보기

`provider.py` 를 보지 않고, `LLM` Protocol(§ 1.2)만 기억한 채로 4번째 구현체를 직접 스케치해보라 — 예를 들어 `class FakeLLM: model = "fake"` 하나에 `generate(self, prompt, *, system=None, max_tokens=4000) → str` 메서드만 채워 넣으면(실제로 `tests/test_chat_agent.py` 가 정확히 이 패턴을 쓴다), 그 클래스는 아무 상속 선언 없이도 Book-forge의 모든 에이전트에 그대로 꽂힌다.

이 질문 하나로 이 장의 핵심을 스스로 테스트할 수 있다. 지금 만들고 있는(또는 만들 예정인) 에이전트가 실제로 호출하는 LLM API를, 이런 최소 Protocol 하나로 추상화할 수 있는가? "예"라고 답할 수 있다면, `provider`를 나중에 바꿀 때(OpenAI→로컬 모델 등) 에이전트 코드를 한 줄도 안 고쳐도 된다.

이 챕터의 핵심

- "AI 에이전트"는 추론 엔진·도구·메모리·오케스트레이션의 조합이며, 자율성은 스펙트럼이다. Book-forge의 개별 에이전트는 그 스펙트럼의 단순한 쪽에 있다 (§ 1.5).
- 에이전트의 최소 계약은 `prompt: str → str` 이다. Book-forge의 `LLM` Protocol이 이를 코드로 명시한다.
- `provider(OpenAI/Anthropic/Ollama)`는 껍데기일 뿐, 에이전트 코드는 `provider`를 몰라도 된다. `create_llm()` 이 환경변수로 그 결정을 한 곳에 모은다.
- 작은 구현 디테일(`think: False`)이 실제 장애로 이어진 사례가 이미 있다. 에이전트를 다룰 때 "모델이 답을 냈는가"를 당연하게 여기면 안 된다.

참고 자료

- `src/book_forge/llm/provider.py` — `LLM` Protocol과 3개 구현체 전체
 - `src/book_forge/config.py` — `load_config()` 가 `.env` 를 읽어 `LLM_PROVIDER` 등을 `os.environ` 에 채우는 경로
 - 부록 A(용어집) — 이후 장에서 낯선 용어가 나오면 언제든지 돌아와 확인할 수 있다
-

다음 챕터는 이 `generate()` 호출 하나가 실제 에이전트(`PlannerAgent`) 안에서 어떻게 프롬프트로 조립되고, 응답이 어떻게 파싱되며, `@agent_eval` 계측이 어느 지점에 끼어드는지 한 줄씩 따라간다.